PUBLIC ARCHITECTURE OVERVIEW

AEGISVERITY

V3.2 (APEX EDITION)

Holistic Zero-Trust Mobile Defense via Offline Hardware Attestation,

Asymmetric Entropy Telemetry, and Deterministic Lockdown

Document Type	Public Architecture Overview (Redacted Edition)
Version	3.2 (Apex Edition)
Classification	PUBLIC — Selected implementation details withheld
Date	March 17, 2026
Full Version	Available under NDA for qualified enterprise partners

Redaction Notice

This document is the public-release version of the AegisVerity V3.2 technical whitepaper.
Sections containing proprietary implementation details, specific detection thresholds,
internal function signatures, and cryptographic parameters have been redacted.
Redacted content is indicated by black bars labeled [REDACTED].
The full unredacted document is available to qualified enterprise partners under NDA.

1. Executive Summary

AegisVerity V3.2 (Apex Edition) represents a paradigm shift in mobile endpoint security, transitioning from reactive, signature-based detection to a proactive, behavioral, and hardware-enforced defense architecture for the Android platform.

The system is engineered to operate under an "Assume Breach" posture, prioritizing impact containment and kill chain disruption over perimeter defense. It functions independently of external cloud dependencies, making it resilient against network-level attacks targeting the security infrastructure itself.

AegisVerity V3.2 addresses three critical threat vectors that defeat conventional mobile security solutions: obfuscated network exfiltration using modern encryption protocols, supply chain and application repackaging attacks, and administrative privilege race condition exploits.

Core Design Philosophy
Zero-Trust across all layers: kernel, network transport, and application lifecycle.
Hardware-rooted trust: security guarantees anchored in Trusted Execution Environment (TEE).
Offline-first: no dependency on external threat feeds or cloud verification APIs.
Assume Breach: focus on containment and forensics, not perimeter prevention.

2. Threat Landscape & Design Motivation

The contemporary mobile threat landscape is dominated by sophisticated surveillanceware (e.g., Pegasus, Predator, Hermit) capable of exploiting zero-day vulnerabilities. Traditional Endpoint Protection Platforms (EPP) face systemic failure against these threats for three fundamental reasons.

2.1 Obfuscated Exfiltration & Protocol Evasion

Modern APTs utilize custom cryptographic tunneling and modern DNS encryption protocols to render malicious traffic indistinguishable from legitimate application traffic. Standard port-based and SNI-based inspection techniques are rendered ineffective against these evasion methods.

2.2 Supply Chain & Repackaging Attacks

Malicious actors may reverse-engineer, modify, and repackage security applications to disable internal protection logic before deployment. A security solution that cannot verify its own integrity is fundamentally untrustworthy.

2.3 Administrative Privilege Exploitation

Attackers possessing physical device access or remote shell access can trivially revoke administrative rights if the security solution does not establish those rights with zero-window locking — meaning no time gap between privilege grant and restriction enforcement.

Threat Vector	Conventional EPP	AegisVerity V3.2
Encrypted C2 Tunneling	Blocked by signature only	Behavioral correlation + entropy analysis
DNS Protocol Evasion	Blind to encrypted DNS	DNS correlation engine (passive)
APK Repackaging	No self-verification	Hardware-rooted integrity chain
Kernel Rootkit	Detected post-infection	Pre-boot TEE attestation gate
Physical Device Access	Disabled by attacker	TEE-backed biometric + auto-lock
ADB Privilege Stripping	Race condition exploitable	Zero-window Genesis Block locking
Background Eavesdropping	OS permissions only	Hardware sensor kill via Device Owner

3. Architecture Overview

AegisVerity V3.2 is structured as a closed-loop defense matrix of six independent modules. Each module addresses a distinct attack surface and communicates with the others through a centralized event bus. The architecture is designed so that the compromise of any single module does not cascade into a full system failure.

Architecture Principle: Defense in Depth
No single module is a single point of failure. Each layer independently
detects and responds to threats within its domain. The Reactor Core serves as
the final enforcement authority, receiving signals from all other modules.

Module A: Hydra Core — Network Telemetry Engine

Hydra serves as the passive network baseline monitor. It continuously measures traffic volumes at both the kernel interface level and the application UID level, identifying statistical discrepancies that indicate hidden or unauthorized data transmission — including traffic generated by kernel-level rootkits that bypass standard application monitoring.

A resilience watchdog mechanism ensures Hydra remains operational even under aggressive system resource management by device manufacturers, preventing denial-of-service attacks against the monitoring subsystem itself.

Module B: The Cortex & The Oracle — Asymmetric Inspection Pipeline

This module is responsible for network traffic analysis. It operates as a two-tier pipeline to balance security thoroughness with performance and accuracy.

The Oracle: Reputation & Correlation Engine (Fast Path)

The Oracle maintains an in-memory reputation cache of known network destinations. Traffic destined for verified, trusted services is passed without deep inspection, eliminating false positives from legitimate encrypted traffic. The Oracle also employs a DNS correlation technique to identify suspicious destinations that attempt to hide behind encryption protocols — without requiring the decryption of any traffic.

The Cortex: Behavioral Entropy Analysis (Slow Path)

Traffic that cannot be cleared by the Oracle is subjected to statistical payload analysis by the Cortex module. The analysis operates at the native code layer for performance and to resist instrumentation attacks. Payloads exhibiting statistical characteristics consistent with double-encrypted or tunneled C2 communication are flagged for Reactor Core action.

Specific detection thresholds and algorithmic parameters are proprietary:

```
[ REDACTED — DETECTION THRESHOLDS & ALGORITHMIC PARAMETERS ]
```

Module C: Obsidian Guard — Native Self-Protection Layer

Obsidian Guard is the application's self-defense module, operating at the native code layer to resist Java-level instrumentation and hooking attacks. It performs continuous integrity verification of the application's own binaries and runtime environment.

Supply Chain Integrity Verification

Obsidian Guard implements a dual-layer verification process that confirms the application has not been tampered with or repackaged since its original signing. The first layer verifies the cryptographic identity of the application's signing authority. The second layer verifies the integrity of the native binary components themselves.

Verification methodology and cryptographic parameters are proprietary:

```
[ REDACTED — CERTIFICATE VERIFICATION METHODOLOGY ]
[ REDACTED — NATIVE BINARY INTEGRITY CHECK PARAMETERS ]
```

Runtime Environment Integrity

Obsidian Guard continuously monitors the runtime environment for indicators of dynamic analysis, debugging, or instrumentation tools. Detection operates across multiple independent techniques to resist bypass attempts that defeat any single method.

Specific detection techniques and signatures are proprietary:

```
[ REDACTED — ANTI-ANALYSIS DETECTION TECHNIQUES ]
```

Upon detection of any integrity violation, the application executes an immediate and unconditional termination protocol. No data is written, no state is preserved, and no user prompt is displayed.

Module D: Identity & Access Management

The IAM module governs all authenticated access to the AegisVerity dashboard and control surface. It is designed to guarantee that an authenticated session cannot persist across application lifecycle transitions — preventing attackers who gain physical access to a device from inheriting a previously authenticated session.

- Session storage is volatile and in-memory only — no session tokens are written to persistent storage
- Session validity is measured using a tamper-resistant monotonic clock, preventing time-manipulation attacks
- Any transition to the application background state results in immediate, unconditional session revocation
- The application dashboard is visually concealed from the Android task switcher

Module E: Smart Gatekeeper — Sensor Access Control

The Smart Gatekeeper enforces Zero-Trust access policies on device sensors (camera, microphone) based on device state. When the device screen is off, the module enforces sensor access restrictions across all non-whitelisted applications using Device Owner administrative authority — not standard runtime permissions, which can be bypassed.

This directly mitigates background audio and video surveillance that exploits the Android OS's permissive default behavior toward background processes.

Module F: Reactor Core — Device Owner Enforcement Engine

The Reactor Core is the final enforcement authority. It receives threat signals from all other modules and executes hardware-level mitigation policies using Android Device Owner (DO) privileges. Its design addresses the key vulnerability of prior DO-based security solutions: the race condition between privilege acquisition and restriction enforcement.

Genesis Block: Zero-Window Privilege Locking

The Reactor Core establishes its administrative restrictions at the precise moment Device Owner privileges are granted by the operating system. There is no time window between privilege grant and restriction enforcement — eliminating the attack surface exploited by race condition attacks against prior DO implementations.

Graduated Response Capability

- Targeted process suspension for identified malicious application UIDs
- Hardware-level sensor disablement at the driver layer
- Forced device lock with biometric re-authentication requirement
- Graceful administrative self-revocation via authenticated failsafe sequence

4. Pre-Flight Integrity & Hardware Attestation

Before any security logic executes, AegisVerity V3.2 performs a hardware-rooted integrity verification of the device environment. This ensures the system does not operate on a compromised host — a scenario where all software-layer security guarantees become meaningless.

4.1 Offline StrongBox TEE Attestation

AegisVerity V3.2 discards reliance on cloud-based integrity APIs that require network connectivity and are therefore vulnerable to network-level attacks. Instead, V3.2 leverages the device's dedicated hardware security module (StrongBox) to generate a cryptographic proof of device integrity that can be verified entirely offline.

If the attestation indicates the device has been compromised at the boot or OS layer, AegisVerity refuses to initialize. Operating on a compromised host would provide a false sense of security.

Specific Root CA identities and verification chain parameters are proprietary:

```
[ REDACTED — ATTESTATION ROOT CA & VERIFICATION PARAMETERS ]
```

4.2 Execution Gate

Device State	Attestation Result	AegisVerity Behavior
Unmodified, verified bootloader	MEETS_STRONG_INTEGRITY	Full initialization proceeds
Hardware-backed key available	MEETS_DEVICE_INTEGRITY	Degraded mode, user notified
Unlocked bootloader / custom ROM	FAILS_INTEGRITY	Refuses to initialize
StrongBox unavailable	Hardware not present	Falls back to software TEE attestation

5. Incident Response & Forensic Logging

Every mitigation action taken by AegisVerity V3.2 is accompanied by a structured forensic record. This supports post-incident analysis, audit requirements, and future integration with enterprise Security Information and Event Management (SIEM) platforms.

5.1 Response Scenarios

Scenario A: Background Exfiltration Attempt

When the device screen is off, any volumetric network anomaly detected by Hydra Core triggers an escalating response: sensor lockdown, process suspension of the offending application, and forced device lock. The incident is recorded with full metadata.

Scenario B: Obfuscated C2 Communication

Traffic failing Oracle reputation checks is subjected to Cortex behavioral analysis. Traffic exhibiting payload characteristics consistent with C2 tunneling is dropped at the VPN interface. The originating process is suspended and the incident is logged.

Scenario C: Physical Access Attempt

Any attempt to access the AegisVerity dashboard after the application has entered the background triggers immediate biometric re-authentication. The prior session has already been revoked. An attacker gains no access to the control surface.

5.2 Audit Log Schema

Each incident record contains the following fields. Specific encoding formats and encryption parameters for the local audit database are proprietary.

Field	Description
Timestamp	Incident detection time (tamper-resistant monotonic clock)
Application UID	Android UID of the process associated with the threat
Network Delta	Volumetric anomaly in bytes (for network-based threats)
Destination	Network destination associated with the suspicious activity
Threat Category	HIGH_ENTROPY / GHOST_TRAFFIC / ENCRYPTED_DNS / PHYSICAL_ACCESS
Action Taken	Mitigation action executed by Reactor Core
Resolution	Whether the threat was successfully contained

[REDACTED — DATABASE ENCRYPTION SCHEME & SIEM EXPORT FORMAT]

6. Security Posture & Capability Summary

Capability	Status	Notes
Pre-boot hardware integrity attestation	Implemented	StrongBox TEE, offline verification
Application supply chain verification	Implemented	Dual-layer, native code layer
Runtime anti-analysis protection	Implemented	Multi-technique, native layer
Encrypted DNS evasion detection	Implemented	Passive correlation, no decryption
C2 behavioral traffic analysis	Implemented	Entropy-based, native layer
Zero-window administrative locking	Implemented	Genesis Block mechanism
Cross-application sensor control	Implemented	Device Owner administrative API
Physical access session protection	Implemented	Volatile sessions + biometric gate
System stability safeguards	Implemented	System app protection logic gate
Forensic audit logging	Implemented	Encrypted local database
SIEM platform integration	Roadmap	Structured log format ready
Multi-user / Work Profile support	Roadmap	Enterprise deployment target

7. Design Ethics & Privacy Principles

AegisVerity V3.2 was developed with an explicit commitment to the principle that security capabilities must not be achieved at the expense of user privacy. This principle directly influenced one significant architectural decision during development.

7.1 Rejected: Forced DNS Plaintext Mode

An approach was evaluated that would have forced all DNS queries on the device to transmit in plaintext, providing the network inspection engine with complete visibility into all domain lookups. This approach was explicitly rejected despite its technical effectiveness.

Why This Approach Was Rejected
Forcing DNS to plaintext would expose all domain queries to ISPs, network operators,
and any observer on the network path — including on hostile public networks.
This would represent a significant reduction in user privacy that extends far beyond
the security monitoring purpose and affects all applications on the device.
The adopted alternative achieves comparable threat detection capability through
passive DNS correlation techniques that do not require disabling existing privacy
protections. Security and privacy are treated as compatible, not competing, goals.

7.2 Privacy Architecture Commitments

- No user traffic content is decrypted or stored — only behavioral metadata
- Audit logs contain network metadata only, not payload content
- Sensor access restrictions protect users from third-party applications, not from the user themselves
- The failsafe mechanism (Ghost Switch) ensures users retain ultimate control over the system
- All security enforcement is transparent to the device owner through audit logging

8. Technical Requirements

Requirement	Specification
Minimum Android Version	Android 8.0 (API 26)
Target Android Version	Android 14 (API 34)
Deployment Method	Android Device Owner provisioning (ADB or NFC bump)
Hardware Requirement	ARM64 processor (arm64-v8a ABI)
StrongBox	Recommended; software TEE fallback available
Network Permission	VPN service permission required for traffic inspection
Administrative Permission	Device Owner privileges required for full functionality

9. Conclusion

AegisVerity V3.2 (Apex Edition) demonstrates that enterprise-grade mobile security is achievable without compromising user privacy, without relying on cloud infrastructure, and without accepting the fundamental weaknesses of conventional signature-based endpoint protection.

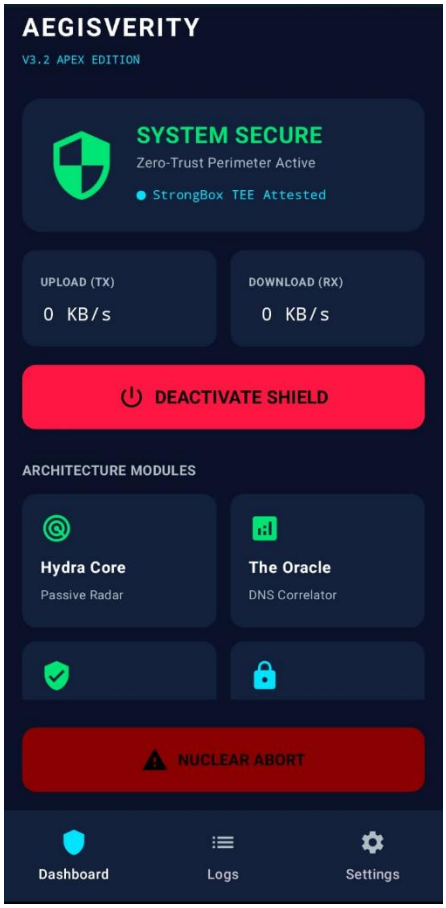
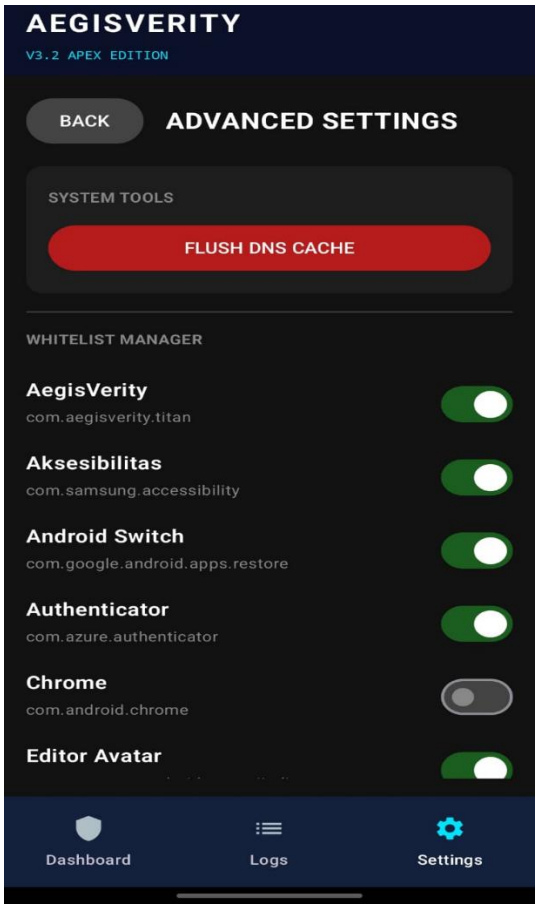
By anchoring trust in hardware rather than software, enforcing Zero-Trust policies at the OS level rather than the application layer, and treating user privacy as a design constraint rather than a trade-off, AegisVerity establishes a new reference architecture for mobile security in high-threat environments.

The architecture has been validated through three iterative design review cycles, with each iteration directly responding to identified architectural gaps. The result is a system whose security properties are grounded in hardware guarantees and deterministic OS-level enforcement rather than probabilistic heuristics.

Availability & Engagement
Proof of Concept: Available for qualified security researchers and enterprise evaluators.
Full Technical Detail: Unredacted whitepaper available under NDA for enterprise partners.
Research Collaboration: Open to peer review and academic security research engagement.
Enterprise Deployment: Contact for Device Owner provisioning and integration support.

Appendix: Redaction Index

The following implementation details have been withheld from this public release to protect proprietary security mechanisms. Disclosure of these details could materially assist adversaries in developing bypass techniques.

Section	Redacted Content	Rationale
3 — Cortex	Detection thresholds & algorithm parameters	Enables adversary payload calibration to evade detection
3 — Obsidian	Certificate verification methodology	Enables targeted bypass of supply chain verification
3 — Obsidian	Native binary integrity check parameters	Enables binary patching that avoids detection
3 — Obsidian	Anti-analysis detection techniques & signatures	Enables instrumentation tools to evade detection
4 — Attestation	Root CA identities & verification chain	Enables forged attestation certificate construction
5 — Logging	Database encryption scheme & SIEM export format	Reduces attack surface on forensic data store

This document is released under the condition that it is reproduced in full without modification. Selective quotation that omits the redaction notice or privacy principles sections is not permitted